

Algorithms

- 1- Print this cover sheet
- 2- Please write all your names in Arabic
- 3- Please make sure that your ID is correct
- 4- Attendance Handwritten Signature should be filled in before discussion
- 5- Please, come on your time. Teams who will be late will subject to -2 of the total grade

- Project Idea:Wiggle Sort.....
- Team Discussion Time:12:20.....
- Team No:73.....

Team Information (Printed by Computer):

Attendance Handwritten Signature	Full Name [IN ARABIC]	ID [Ordered by ID]	
	ابراهيم احمد محمد عبد السلام	20240008	1
	حمزة عبدالمجيد محمد	20240313	2
	عبد الله محمد مهني حسن	20240582	3
	عمر محمد صلاح الدين عبد الغفار	20240643	4
	عمر مصطفى الحسيني شاهين	20240657	5
	فارس اسماعيل عبد المعطي غانم	20240680	6
	كريم احمد ابراهيم	20240703	7

Project Requirements

First algorithm (non-recursive)	Pseudo code	1	
	Implementation	2	
	Analysis & complexity (steps)	2	
Second algorithm (recursive)	Pseudo code	1	
	Implementation	2	
	Analysis & complexity (steps)	2	
Comparison	1		
Understanding	4		

Wiggle Sort

Algorithm Analysis Report

Problem Description

Given an integer array `nums`, reorder it such that:

`nums[0] < nums[1] > nums[2] < nums[3] ...`

The input array is guaranteed to always have a valid answer.

First Algorithm (Non-Recursive)

Pseudocode

The non-recursive wiggle sort uses an iterative loop to place elements:

wiggleSortNONREC(nums)
1. <code>sorted ← copy of nums</code>
2. <code>CALL mergeSort(sorted, 0, n-1)</code>
3. <code>n ← length of nums</code>
4. <code>left ← (n - 1) / 2</code>
5. <code>right ← n - 1</code>
6. <code>FOR i ← 0 TO n - 1 DO</code>
7. <code>IF i is even THEN</code>
8. <code>nums[i] ← sorted[left]</code>
9. <code>left ← left - 1</code>
10. <code>ELSE</code>
11. <code>nums[i] ← sorted[right]</code>
12. <code>right ← right - 1</code>
13. <code>END IF</code>
14. <code>END FOR</code>
15. <code>RETURN nums</code>

MergeSort Pseudocode:

mergeSort(arr, left, right)
1. <code>IF left >= right THEN RETURN</code>
2. <code>mid ← left + (right - left) / 2</code>
3. <code>CALL mergeSort(arr, left, mid)</code>
4. <code>CALL mergeSort(arr, mid + 1, right)</code>
5. <code>CALL merge(arr, left, mid, right)</code>
<code>merge(arr, left, mid, right):</code>

1.	Copy arr[left..mid] into L[]
2.	Copy arr[mid+1..right] into R[]
3.	$i \leftarrow 0, j \leftarrow 0, k \leftarrow \text{left}$
4.	WHILE $i < n_1$ AND $j < n_2$ DO
5.	IF $L[i] \leq R[j]$: arr[k] $\leftarrow L[i]$; $i++$
6.	ELSE: arr[k] $\leftarrow R[j]$; $j++$
7.	$k++$
8.	Copy remaining elements of L[] into arr[]
9.	Copy remaining elements of R[] into arr[]

Analysis & Complexity

Step-by-step complexity analysis:

Step	Operation	Cost	Explanation
1	Copy array	$O(n)$	Create a duplicate of nums
2	MergeSort (sort copy)	$O(n \log n)$	Divide-and-conquer sort
3	Iterative placement loop	$O(n)$	Single pass, n iterations
4	Space for L[] and R[] arrays	$O(n)$	Auxiliary space used in merge

Complexity	Value	Reason
Time	$O(n \log n)$	Dominated by MergeSort step
Space	$O(n)$	Sorted copy + merge temp arrays

Second Algorithm (Recursive)

Pseudocode

The recursive wiggle sort delegates element placement to a recursive helper function:

wiggleSortREC(nums)
1. sorted $\leftarrow$ copy of nums
2. CALL mergeSort(sorted, 0, n-1)
3. $n \leftarrow$ length of nums
4. left $\leftarrow (n - 1) / 2$
5. right $\leftarrow n - 1$
6. CALL recWiggleSort(nums, sorted, left, right, i=0)

recWiggleSort(nums, sorted, left, right, i)	
1.	IF i == n THEN RETURN // base case
2.	IF i is even THEN
3.	nums[i] ← sorted[left]
4.	left ← left - 1
5.	ELSE
6.	nums[i] ← sorted[right]
7.	right ← right - 1
8.	END IF
9.	CALL recWiggleSort(nums, sorted, left, right, i + 1)

Analysis & Complexity

Step-by-step complexity analysis:

S t e p	Operation	Cost	Explanation
1	Copy array	$O(n)$	Create a duplicate of nums
2	MergeSort (sort copy)	$O(n \log n)$	Same merge sort used in both versions
3	Recursive placement calls	$O(n)$	n recursive calls, one per element
4	Call stack space	$O(n)$	Each call adds a frame to the stack

Complexity		Value	Reason
Time		$O(n \log n)$	Dominated by MergeSort step
Space		$O(n)$	Sorted copy + call stack depth of n

Comparison

Aspect	Non-Recursive	Recursive
Time Complexity	$O(n \log n)$	$O(n \log n)$
Space Complexity	$O(n)$	$O(n)$ — includes call stack
Placement Method	Iterative for loop	Recursive helper function
Stack Overflow Risk	None	Yes — for very large n
Readability	Straightforward, easy to trace	Clean base case, elegant structure

Practical Performance	Slightly faster (no function call overhead)	Slightly slower (call stack management)
Best Use Case	Large inputs, production code	Small inputs, educational / clear code

Both algorithms produce the same correct wiggle-sorted output and share the same asymptotic complexity. The key difference is implementation style: the non-recursive version uses a simple loop and is safer for large inputs, while the recursive version is more elegant but risks stack overflow for very large arrays.